

The Cost of Speculation: Revisiting Overheads in the V8 JavaScript Engine

Alberto Parravicini*
DEIB, Politecnico di Milano
Milano, Italy

alberto.parravicini@polimi.it

* Work done while at Huawei Technologies, Switzerland.

Rene Mueller
Computing Systems Laboratory
Zürich Research Center, Huawei Technologies Switzerland
rene.mueller@huawei.com

Abstract—JavaScript, while already widely used in web applications, is also gaining popularity beyond the web, such as on the server as part of Node.js or on the desktop with the Electron framework (e.g., in the Slack application). However, executing code written in this weakly and dynamically typed language efficiently, requires sophisticated Just-in-time (JIT) compilation. In this paper, we revisit the execution overheads of JavaScript. We perform a detailed analysis of the JetStream2 benchmark suite on Google's modern V8 JavaScript engine. We identify micro-architectural bottlenecks and runtime overheads that result from the speculations made by the JIT compiler when it generates machine code from JavaScript. We find that checks that verify assumptions made by the compiler have an average execution overhead of 8 %, 2–4x of what an earlier study reported on an older version of V8. For the check for Small Integers (SMI), we observe that the conditional branches are not the underlying cause but rather the computation of the condition itself. This indicates that these checks provide an attractive avenue for a HW/SW codesign solution. We present an extension for the ARMv8 instruction set that optimizes SMI load instructions and checks. We can improve execution time on SMI-heavy computations by up to 10 % on a prototype implementation of the new instructions in the gem5 simulator.

Index Terms—JIT Compilation, JavaScript, Performance Evaluation

I. INTRODUCTION

According to reports [1], [2], there are 1.8 billion websites and more than 97 % use JavaScript as the client-side programming language. However, JavaScript is not only running inside the browser but is also used for server-side applications, e.g., in Node.js and Deno runtimes. Furthermore, JavaScript is also getting increasingly popular in desktop applications. Applications written in JavaScript, HTML, and CSS—built on top of the Electron framework—can run across different desktop platforms. The most prominent Electron applications are Microsoft Visual Studio Code and the Slack app.

JavaScript, officially standardized as ECMAScript ECMA-262, is a dynamically and weakly typed object-oriented programming language. Although this type system provides the flexibility of writing generic code, as shown in this function,

```
function add(a, b) {
  return a + b
}
```

it also adds to the complexity of the language runtime, specifically, the Just-In-Time (JIT) compilation to machine code. The function can be called with *a* and *b* of any type. The ECMAScript specification defines double-precision floating-point as number format. However, if *a* and *b* are both 32-bit integers, there is no need to use double-precision floating-point addition. The operation can be performed as a 32-bit add. On a Cortex A76 core, for example, an integer addition has half the latency of a floating point addition. If *a* and *b* are strings, the addition is a string concatenation. The weak typing semantics in JavaScript increases the complexity further. For example, if *a*

is an array [1, 2, 3] and *b* is the integer 7, the ECMAScript specification states that the array object and the number are first to be coerced into strings "1,2,3" and "7", respectively, and then concatenated, yielding the result string "1,2,37".

JavaScript engines, such as V8 (Google), JavaScriptCore in WebKit (Apple), or SpiderMonkey (Mozilla), execute JavaScript code through a series of tiered compilation steps. Code is initially executed by an interpreter. The interpreter collects dynamic type information for object accesses and operations, such as *a+b* in the previous example. This type information is later used during compilation to generate efficient machine code. For example, if *a* and *b* were always found to be integers during interpretation, the JIT speculates that this will also be the case in the future and generate machine code for integer addition. Of course, this assumption may not hold. The JIT therefore needs to insert checks that verify the assumption. If they are no longer valid, the execution of the JIT-compiled code needs to stop and may fall back to the interpreter (also called deoptimization or bailout). The code segment needs to be re-compiled. These dynamic checks are called *deoptimization checks* in the following. The checks are not free: 4–5 % of instructions in compiled code, both on x86_64 (we refer to it as X64) and ARM64, are deoptimization checks (Figure 1).

In this work, we analyze the impact of these checks on the overall runtime overhead and the implications on the micro-architectural state of the processor. Previous studies on this topic [3], which estimated a 2–4 % overhead, are likely outdated, given the continuous evolution of V8 and the increasing popularity of ARM64 CPUs, which were not considered before. We also observe an increasing interest in HW/SW codesign optimizations for dynamic languages [4], [5]. On this topic, we investigate how we can reduce check overheads through a custom ARM64 Instruction Set Architecture (ISA) extension.

We make the following contributions:

- We study the JetStream2 benchmark suite that consists of 51 JavaScript benchmarks on the V8 JavaScript engine (Section III).
- We analyze the runtime overhead from checks introduced by the V8's TurboFan JIT when it speculates on types and discuss their micro-architectural effects (Section IV).
- We create an instruction set extension for ARMv8 that optimizes one of the most frequent checks, the Small-Integers (SMIs) check, using custom memory instructions (Section V-A).
- With the gem5 micro-architectural simulator, we show that this ISA extension can reduce the execution time by up to 10 % on SMI-heavy workloads, and by 3 % on average (Section V-B).

II. BACKGROUND

This section provides the necessary background on the inner workings of a modern JavaScript engine (Section II-A). While we focus on V8, other engines (e.g., JavaScriptCore/WebKit and

checks such as *Lost-precision* and *Division-by-zero*. Among the most common and time-consuming deoptimization checks we find:

- 1) *SMI and Not-a-SMI*: V8 can use compressed pointers on 64-bit platforms. These pointers are stored as tagged 32-bit values that can also represent Small-Integers (SMIs). The Least-significant Bit (LSB) is the tag. If this tag bit is cleared, the remaining bits are a signed 31-bit integer. If it is set, the remaining bits are a tagged pointer. SMI values can be stored directly, without requiring a heap access. This memory optimization is critical to avoid memory accesses. However, it requires bit-shifts to turn SMIs into machine integers (i.e., an *untagging right-shift*), and checks that inspect the value's LSB.
- 2) *Boundary Checks*: these checks ensure that array accesses are within the array's bounds. Array indices are often stored as SMIs, which require additional checks and untagging shifts.
- 3) *Wrong Map Checks*: JavaScript is prototype-based language. Properties can be dynamically added and removed from individual objects. V8 uses hidden classes, called maps, to handle objects of the same shape. Each object shape encountered at run time is assigned a map. Maps act like classes in that they keep the storage offsets of their objects' properties. The objects themselves have a 32-bit tagged pointer to their map object. To guarantee that an object has the expected shape when used in compiled code, V8 checks that the object's map pointer points to the same map the compiler expected.

This list is not exhaustive. There are additional checks, which we omit here as they are less frequent, or self-explanatory (e.g., overflow checks). Our method can be applied to other JavaScript engines and dynamic languages that use similar checks. We pay special attention to *Not-a-SMI* checks, given their abundance and since SMIs also appear in connection with other checks such as *boundary checks*. As we will show in Section V, *Not-a-SMI* checks are a strong candidate for performance improvements through HW/SW codesign. V8 can be built with 32-bit SMIs instead of 31-bit SMI by setting the corresponding compile-time flag. The flag is set when pointer compression is disabled. Node.js uses 32-bit SMI whereas Chromium, Deno, and our experiments with the standalone V8 use 31-bit SMIs. 32-bit SMIs still use the LSB for tagging, and require the same deoptimization checks and *untagging shift*. Therefore, our results do not depend on the chosen SMIs representation.

C. The Extended JetStream2 Benchmark Suite

Our analysis of overheads of the deoptimization checks focuses on the JetStream2 suite [12], which is maintained as part of Apple's WebKit JavaScript engine. JetStream2 is a large and heterogeneous collection of benchmarks that allow us to tests V8 from different angles, including mathematical computations, regular expression, and language parsers. It also includes most benchmarks that can be found in older suites, such as Octane [13] (used in Southern et al. [3]), Kraken [14] and SunSpider [15]. We exclude the WebAssembly benchmarks of the JetStream2 suite from our analysis as they are not relevant for our study. WebAssembly is statically typed and, therefore, does not require speculation on types for efficient execution. We grouped JetStream2 benchmarks by the language feature they stress (e.g., *string manipulation*) or by their application domain (e.g., *cryptography*). Benchmarks within the same category exhibit similar overhead characteristics. We add six custom sparse linear algebra kernels used to test JavaScript performance in memory-intensive computations with many indirect memory accesses. One is CSR Sparse matrix-vector multiplication (SpMV), which we test

7	ldr x5, [sp, #88]	load array index
0	tst w5, #0x1	test if SMI
1	b.ne #+0x232	deopt if not-a-SMI
1	asr x5, x5, #1	shift SMI to machine-int
13	ldur w4, [x2, #11]	load array size
2	asr x4, x4, #1	shift SMI to machine-int
2	cmp x5, x4	out-of-bounds check
1	b.hs #+0x224	deopt if out-of-bounds
0	lsl x6, x5, #2	prepare index for array access
0	add x4, x6, #0x7	
10	ldr w3, [x3, x4]	load operand through array access
12	ldr x4, [sp, #80]	load operand from stack
2	tst w4, #0x1	test if SMI
1	b.ne #+0x208	deopt if not-a-SMI

Fig. 3: PC samples collected by perf on an ARM64 code fragment containing two SMI checks and one boundary check (in orange).

for different data types (floating-point, large integers and SMI), to capture the performance difference of type-dependent checks.

In Section V, we validate our analysis with a set of SMI-intensive benchmarks, some of which have been adapted from JetStream2 using cycle-accurate simulation in gem5. Due to the complexity of the JetStream2 suite and the resulting simulation overhead, we are simulating a subset of the workload that we run on real hardware.

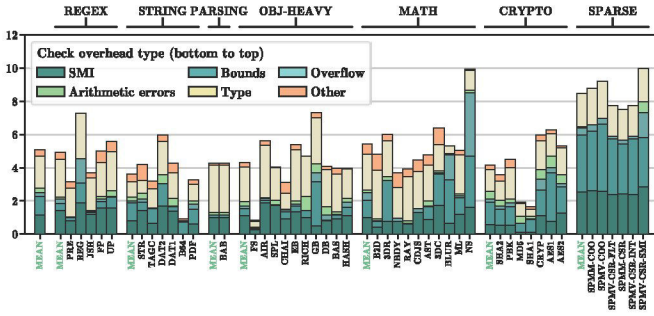
D. Experimental Setup

We conduct our tests on V8 9.2.0 through its D8 shell, built using default release settings. Tests are performed on an X64 Intel Xeon E5-2680 v2 @2.80 GHz with 384 GB of RAM, and an ARM64 TaiShan 2280 server with Kunpeng-920 out-of-order cores and 64 GB of RAM. We ran each benchmark of the suite for 1,000 iterations and repeated this process 30 times.

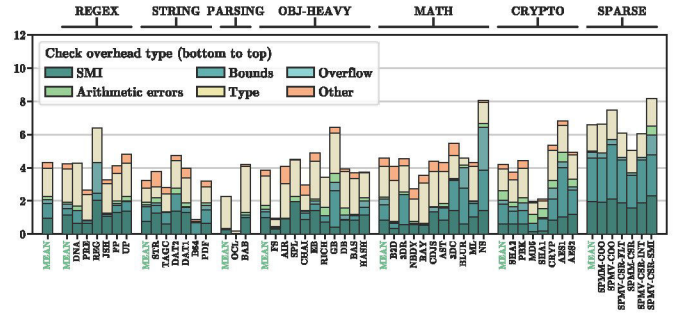
III. CHARACTERIZATION OF CHECK OVERHEADS

Quantifying the runtime costs of the deoptimization checks is not trivial due to the complexity of the V8 engine. One possible approach is instrumenting the generated code, followed by measuring or analyzing the generated machine code, and then estimating the costs through a performance model. However, instrumenting TurboFan, the V8 compiler, could skew results as it would also affect the compilation process itself; similarly, techniques based on statistical sampling might not capture the state of multiple in-flight instructions in Out-of-Order (O3) CPUs. To address these challenges, we measure the check overhead with a methodology that applies statistical inference to the estimates of both techniques, providing greater robustness and deeper insights than previous techniques.

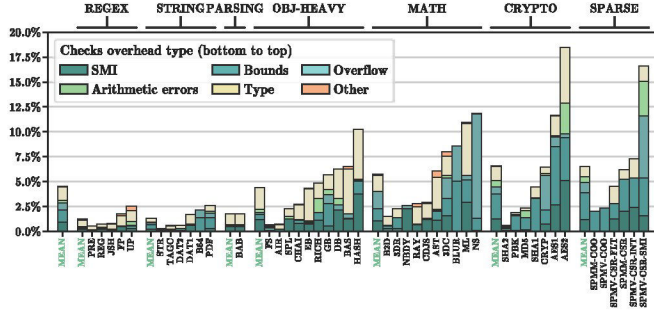
For comparison, Dot et al. [6] estimate the overheads of checks by introducing additional machine instructions (e.g., a counter) in JIT-compiled code. They measure between 10% to 20%, on a simulated O3 desktop core across the Octane, Kraken, and SunSpider benchmark suites. They approximate overheads by counting how many times checks are executed relative the total instruction count. This approach is precise enough, as it does not account for the individual cost of instructions. Southern et al. [3] modify the V8 compiler to perform a coarse-grained elimination of checks during compilation. Compared to them, we use two orthogonal methods to estimate the overhead of the individual checks. The first method (Section III-A) estimates check overheads from time-based Program Counter (PC) sampling. The second method directly measures the impact of checks on the execution time. We can modify the TurboFan compiler to remove either the check branches or the checks altogether, where removing them does not alter the benchmarks' correctness (Sections III-B and IV-B). The advantage of the former method is that overhead can be simply estimated from a profiler trace. In contrast,



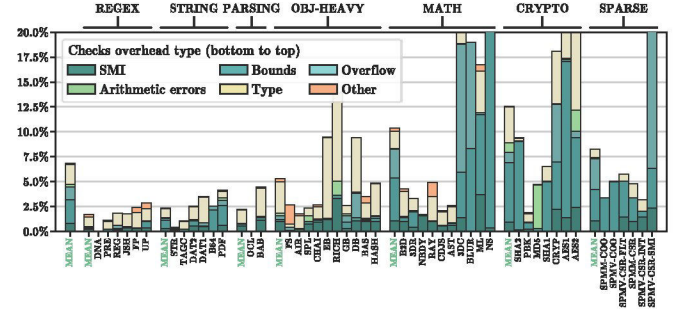
(a) Number of checks per 100 iterations, on X64.



(b) Number of checks per 100 iterations, on ARM64.



(c) Time breakdown of checks, on X64.



(d) Time breakdown of checks, on ARM64.

Fig. 4: Breakdown of the number of checks and their overhead, on x64 and ARM64. We estimated overheads on X64 by considering one instruction before the check’s branch as part of the check, on ARM64 the two previous instructions. Checks on ARM64 require more instructions due to the RISC nature of the ISA. Benchmarks are ordered by overhead, using x64 as reference.

the latter directly measures the overhead, although it ignores the overhead of checks that we leave in for correctness.

A. Check Overheads from Program Counter Sampling

We use `perf` to sample the PC in JIT-compiled code and count the samples that fall on instructions that are part of a check. Figure 3 shows an example of a sequence of machine instructions with the corresponding sample count. We can identify checks because deoptimization paths always jump to a specific region at the end of a compiled function. In addition, because every check has a different target address, we can also determine its type with the help of the annotated assembly listing provided by V8.

Checks do not only consist of branches alone. Other instructions that determine the branch condition, such as `cmp` or `test`, must also be counted as part of the check. For example, in boundary checks, the size of the array must be loaded into a register and compared with the index (Figure 3). Identifying which instructions are part of the check, though is not straightforward. For example, the result of an instruction may be used in the check as well as in regular code and, thus, should not be considered overhead. Furthermore, the instruction scheduling done by the compiler interleaves instructions that are part of a check with main-line instructions. As a pragmatic heuristic, we consider instructions within a certain window preceding the deoptimization branch as part of the check. On the CISC X64 ISA, we consider just one instruction before the branch. On ARM64, we use two instructions before the branch as this RISC ISA requires more instructions to compute conditions with memory operands. We observe that a window size of two aligns best with the exact overhead measurements obtained through the second method (Section IV). Once we identified the instructions that are part of a check, we assign per-instruction sample count from `perf` counts to each check. The

overhead of checks is the ratio between the PC samples identified as part of a check and the total number of collected PC samples. The advantage of this approach is its non-intrusiveness. No engine changes are necessary, and execution time is only affected by the overall profiling overhead.

Distribution of Check Types: Figure 4 depicts the breakdown of the different check types in the benchmarks. Figures 4a and 4b show how many checks TurboFan emits per 100 machine instructions on X64 and ARM64, respectively, while Figures 4c and 4d show the overheads for each check type and benchmark. The frequency varies between 2 to 10 with an average of 5 checks for 100 instructions, coherently with previous studies [3]. In contrast, the overhead of checks in JIT-compiled code is higher, ranging between 5% and 7%, compared to older estimates of 2–4%. Type checks are the most common, with almost half of the occurrences. However, their actual overhead is significantly lower, accounting for only 30% of the total. Type checks dominate only in objects-heavy benchmarks. The next frequent checks are *SMI*, *Not-a-SMI* and *Boundary Checks*. Together, they account for 50% of both frequency and overhead. We target our ISA extension at SMI arithmetic Section V. It not helps *SMI* and *Not-a-SMI* checks but also *Boundary Checks*, since they also contain SMI operations (e.g., on SMI indices).

The frequency is lower (2–8 checks for 100 instructions) on ARM64 than on X64. The reason is that the JIT needs to emit more instruction for the RISC architecture than for X64. Furthermore, the absolute number of checks does not depend on the architectures; the same number of checks are distributed over more instructions resulting in a lower frequency.

Sparse Linear Algebra benchmarks have the highest check frequency because they are simple benchmarks with relatively few instructions. However, their checks have a moderately low overhead.

The only exception is SpMV on SMIs, which requires many overflow checks in arithmetic operations that have result type SMI.

Compared to the analysis in [3], we spot differences that corroborate the need to re-evaluate the impact of checks in V8. Among others, CRYPT no longer has 50 % of overflow checks (it has almost none), Navier-Stokes (NS) has only 10 checks per 100 instructions (instead of 20), and splay (SPL) has now 4 times as many checks per 100 instructions.

Overheads: The distribution of overheads is highly dependent on the benchmark category. In general, mathematical and crypto benchmarks have the highest overheads. This can be explained by the high number of boundary, SMI, and overflow checks. Regular Expression benchmarks on the other hand do not have any significant check overhead, which is also expected because most of their computation is performed by Irregexp, V8’s regex engine, and not in JIT-compiled code. String Manipulation and Language Parsing benchmarks also have low overheads, possibly due to the heavy use of builtin functions that do not require checks (e.g., string concatenation or equality). For the Richards (RICH) benchmark, which tests the performance of object property accesses, we can observe significantly higher overheads due to type checks on ARM64 than on X64, highlighting the difference between the two ISAs again: some checks, such as *wrong map* and *boundary checks*, require multiple instructions on ARM64, while X64 can perform them with fewer (or possibly just one) instructions, resulting in lower overheads for these check classes.

B. Removing Checks in the V8 JIT

Sampling how often a given instruction appears in the PC trace does not necessarily reflect its true cost, especially in O3 CPUs. In order to increase the accuracy of the cost estimation, we combine the sampling technique with another approach that removes checks in JIT-compiled code altogether by modifying the V8 compiler itself. To effectively suppress the emission of checks during the JIT-compilation stage in V8, it is not sufficient to disable the generation of machine instructions that are involved in the checks. The reason is that while the program is lowered from the Sea-of-nodes (SoN) graph to instructions, the provenance of a given instruction is lost—it is no longer known whether an instruction belongs to regular code, a check, or both. As a more robust approach, thus, we operate directly on the SoN representation. We short-circuit deoptimization condition with a Boolean constant (0), as shown in Figure 5. This way, we not only remove the nodes representing checks in subsequent graph optimization passes but also all ancestor nodes that are only used in the checks (for example, loading the array size to perform out-of-bounds array accesses checks), as they are marked as dead code. We can selectively disable individual check types, which is useful when testing the overhead of one specific check or to prevent changes in the program semantics when checks get actually triggered.

1) **Removing Checks:** We further split the costs in the checks into the computation of the Boolean check condition and the conditional branch that follows. To this end, we modified the late code-generation portion of TurboFan to generate the check condition (e.g., the bit-test for SMIs) without emitting the conditional branch instruction. This allows us to determine whether the additional control-flow complexity has an observable performance impact. We find that omitting the deopt branches reduces the committed branches by 20 %. However, the speedup is hardly noticeable with 1–2 % on average. We discuss check branches in more detail in Section IV-B.

2) **Leftover checks:** 16 benchmarks out of 51 do not complete execution correctly if all checks are removed, since they trigger some

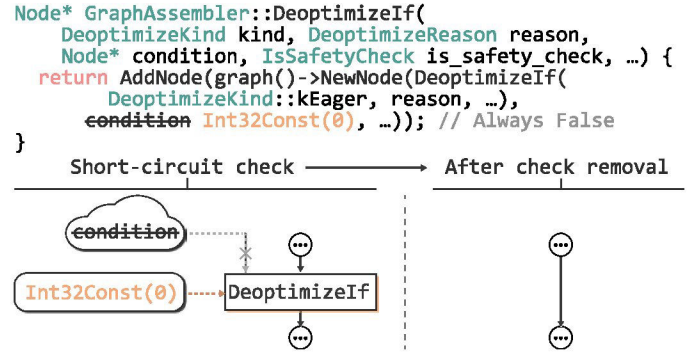


Fig. 5: To prevent the insertion of checks, we short-circuit the node by replacing the original condition with an always-false constant. This node, the original condition and the deoptimization node are marked as dead code and deleted from the Sea-of-Nodes graph.

deoptimizations as part of their normal execution flow. Therefore, we are removing only those check types in a benchmark that will not fail to guarantee correct execution. With this method, less than 20 % of checks of the otherwise failing benchmarks remain in the code. This leftover overhead, estimated from perf, is less than 0.5 %. Overall, the amount of remaining checks is small enough to allow precise measurements through the check removal technique in this section.

3) **Execution time profile with and without checks:** We measure the execution time of the JIT-compiled code for each iteration of the benchmark run. V8 progressively compiles code as it executes, reducing the execution time for each iteration. Figure 6 shows the execution time, relative to the first iteration, for 1,000 iterations of each benchmark. The two lines depict the time with checks and after removing them. The difference between the lines indicates the check overhead. We also include the 16 benchmarks (marked with *) for which it was not possible to remove all checks without altering the semantics. Vertical bars indicate the occurrence of deoptimization events. As the figure shows, these events are very rare and typically occur within the first few iterations of each benchmark. Therefore, further optimizations of the bailout path will not provide any speedup once the generated code reaches steady-state. However, code without checks is significantly faster, with a mean overall time difference of 8 %. If all leftover checks were removed, this estimate would be higher. This value is 2–4× higher than the estimates from Southern et al. [3], and slightly larger than the one obtained in Section III-A through statistical sampling. Therefore, investments in future optimizations should reduce the overhead of the checks rather than improve the deoptimization path. This motivates our ISA extension.

From Figure 6, we identify outliers such as the Crypto-AES-SP (AES2) and sparse benchmarks, which are more than 30 % faster without checks. With checks, SpMV on SMI values are even 20 % slower than the large integer and floating-point counterparts, even though 31-bit integer arithmetic has the lowest complexity conceptually. This difference is due to the checks required in SMI arithmetic to detect overflows and other unexpected behaviors, as seen in Figure 4.

IV. STATISTICAL ANALYSIS OF CHECK OVERHEADS

We perform further statistical analyses of the overhead estimates of the deoptimization checks obtained through profiling in Section III-A and through removal of checks in Section III-B by combining both

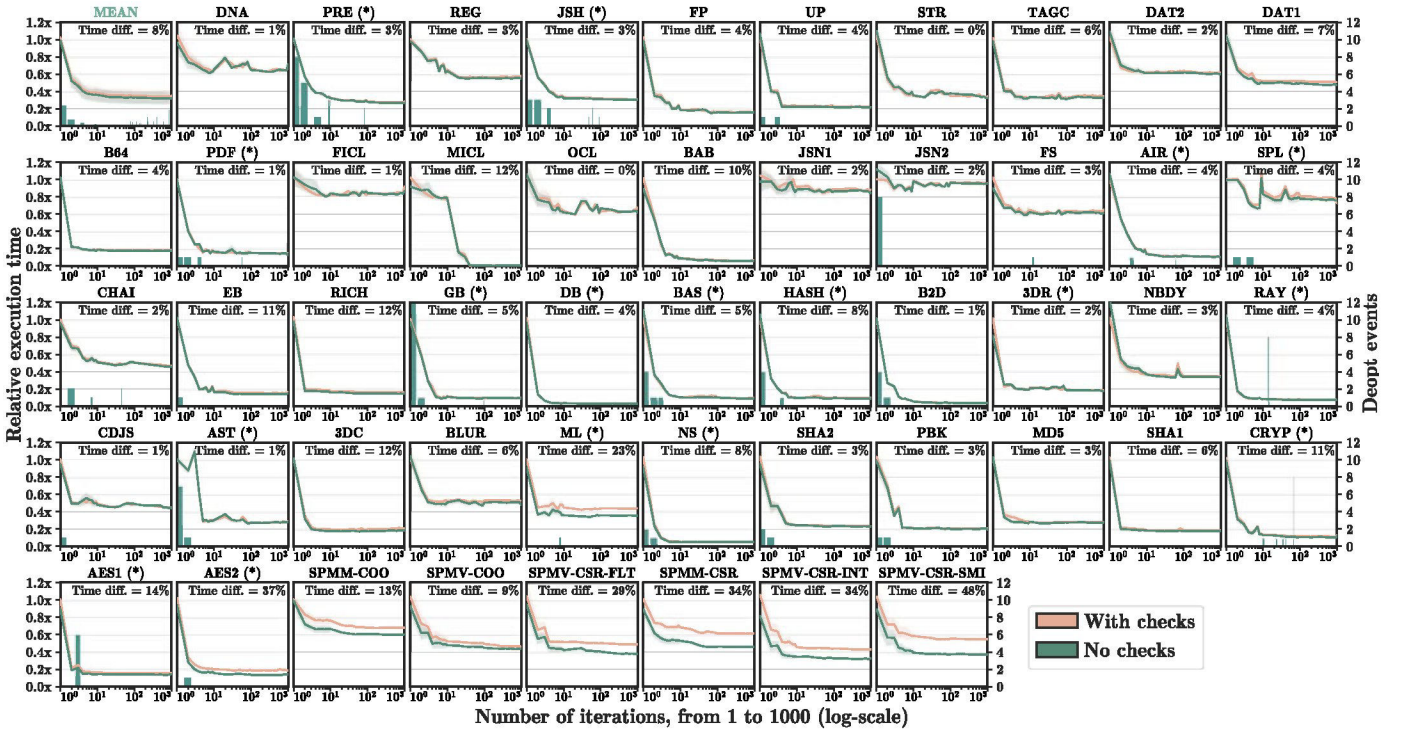


Fig. 6: Relative execution time per iteration with respect to the first iteration, with checks and after removal of checks. Vertical bars indicate the number of deoptimization events. Deoptimizations are rare. Most deoptimizations happen in the first 10 iterations. Compiled code in steady-state is also 2.5x faster than unoptimized interpreted code. Benchmarks with leftover checks are marked with (*).

techniques. We also characterize deoptimization branches and show how they have a minor but not negligible performance impact.

A. Statistical Analysis of Check Overhead

To better understand the relationship between our overhead estimates, we compare measurements obtained from PC-sampling and check removal for each category (Figure 8) and for individual benchmarks (Figure 7). Since removing checks reduces the execution time, we convert the estimates of our two techniques into a speedup to directly compare them. In the case of PC-sampling, we obtain the estimated speedup as $(1 - \%_{ovh}/100)^{-1}$, where $\%_{ovh}$ refers to the check overhead values in Figure 4. For our second technique, the relative execution time differences from Figure 6 can directly be interpreted as speedups.

We start our analysis with speedups grouped by category, in Figure 8. For many benchmark categories, the speedups estimated by PC sampling and check removal are similar. However, there are cases where our two techniques provide different results, such as in sparse benchmarks on x64 and mathematical benchmarks on ARM64. This difference justifies a deeper statistical analysis of our estimates to understand whether speedups are only separated by a proportionality factor and, thus, can still be considered correlated.

In Figure 9 each benchmark is represented as a point whose coordinates are the speedup values estimated by our two techniques, one for each axis. We also estimate the joint density distribution (in red) and show the linear regression with 95 % confidence intervals. We observe non-negligible correlations between the estimates: $R^2 = 0.51$, 71 % correlation on x64, and $R^2 = 0.36$, 60% correlation on ARM64. Most importantly, a statistical test whose null hypothesis is a zero correlation gives close-to-zero p-values in both cases ($< 10^{-6}$), showing a statistically significant correlation. The lower correlation

on ARM64 is explained by the complex structure of checks in this RISC ISA. These results demonstrate the robustness of our overhead estimation methodology and highlight how we can combine information from two very different approaches to obtain a coherent picture of the performance of one of the most complex parts of V8.

When comparing individual speedup estimates in Figure 7, we see how results can significantly change between benchmarks and ISAs. Some benchmarks show hardly any performance improvement; others are more than 20 % faster. Moreover, there is some non-determinism in V8 in how JIT-compilation and garbage collection are triggered. This introduces noise in our measurements as shown by the 95 % percentile error bars in Figure 7. As real-world applications can also have noisy and unpredictable performance, we against attempting to eliminate the noise through an artificially setup (e.g., by thread pinning or fixating garbage collection and JIT times), but rather follow a thorough statistical analysis. We, therefore, conduct individual hypothesis tests to identify which benchmarks have a statistically significant overhead that can be attributed to deoptimization checks.

Practical significance of check overheads: In Figures 7 and 9, we identify the practically significant benchmarks, i.e., we separate benchmarks with noisy results from those benchmarks that present a real performance difference that can be attributed to deoptimization checks. In this context, the relative execution times of each benchmark, with or without checks, represent a population. We use the total execution duration of the benchmark since repeating the analyses on the steady-state (e.g., the last 300 iterations) did not reveal any evident difference. The significance of tests is set to 0.05, adjusted with Bonferroni's correction over the number of tests [16]. We identify practically significant benchmarks as benchmarks with a statistically significant performance difference $\geq 2\%$. In other words, we can

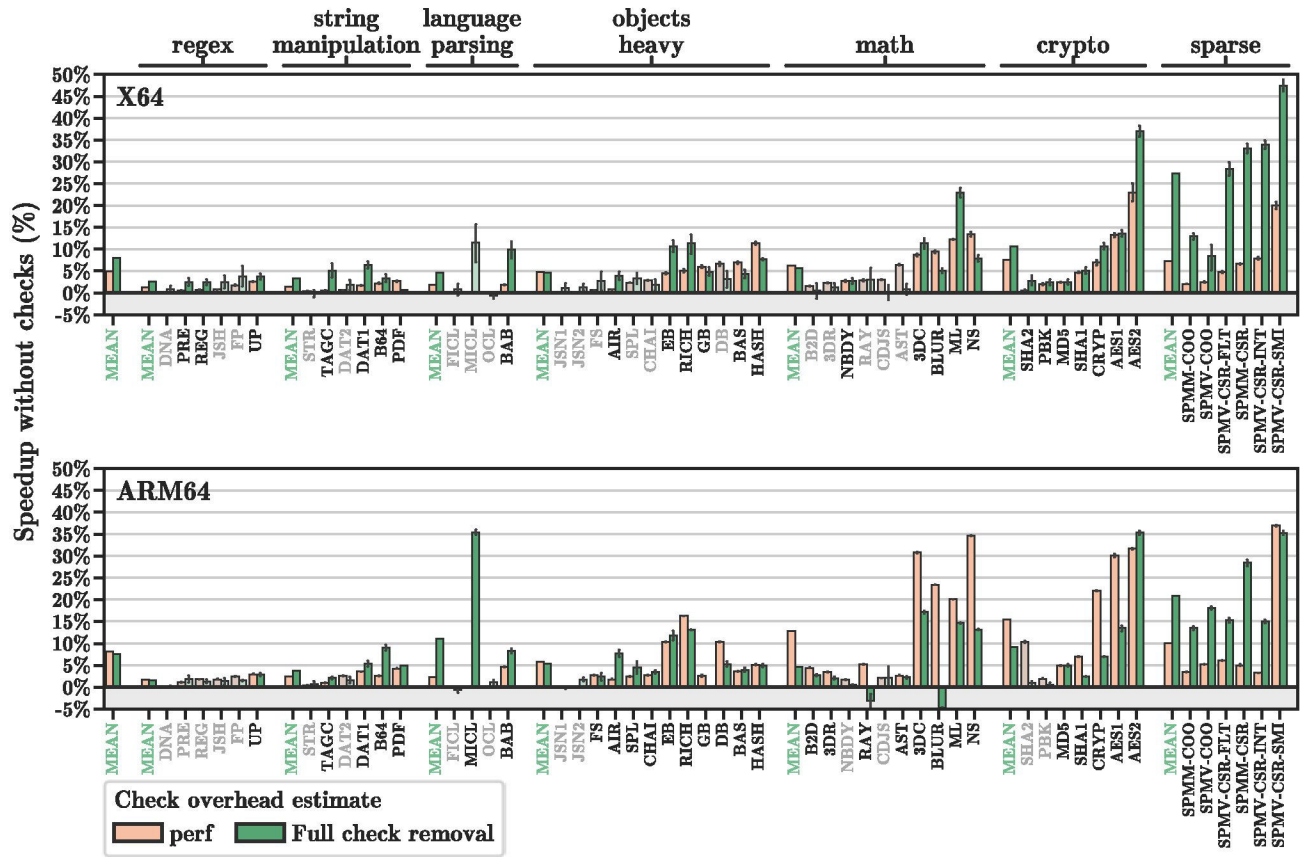


Fig. 7: Check overheads estimated by `perf` and check removal, showing individual benchmarks on X64 and ARM64. Gray labels denote benchmarks where the performance difference is not practically significant (see Section IV-A).

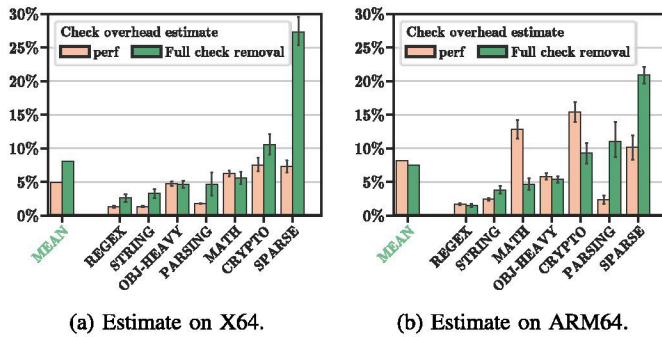


Fig. 8: Deoptimization check overheads, averaged by benchmark category, on X64 and ARM64. Trends are similar, with estimates ranging from 5% to 10% on average.

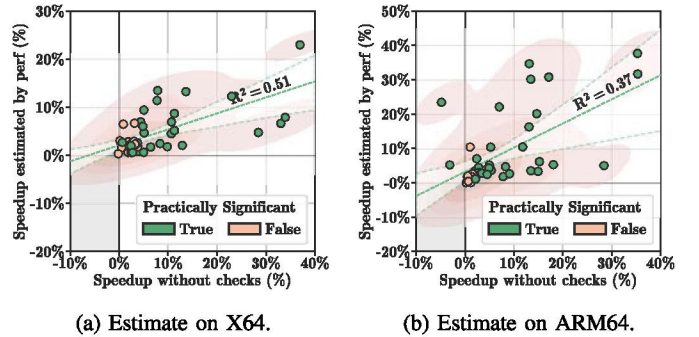


Fig. 9: Correlation of check overhead estimates obtained with PC-sampling and check removal. We measure a statistically significant positive correlation in both cases.

safely ignore benchmarks with a performance difference below 2 %, even if statistically relevant, as their performance gap is too narrow to be meaningful. We divide populations between normally and non-normally distributed, using the D’Agostino-Pearson normality test [17]. Then, we employ the t-test for independent samples on normally distributed populations, adjusted to Welch’s test if the populations variance is not the same. The Wilcoxon signed-rank test is applied to non-normally distributed populations [18].

After performing the statistical significance tests, we can conclude that 31 out of 51 benchmarks (60 %) on X64 and in 34 benchmarks

(67 %) on ARM64 show performance improvement when checks are removed, despite the noisy measurements. Most benchmarks that do not gain a significant speedup contain only a few checks (e.g., benchmarks with parsing or regular expression). However, the Multi-Inspector-Code-Load (MICL) benchmark, a cache-intensive parsing benchmark, is 35 % faster on ARM64. While on X64, it also had a large speedup, it had too much variance to be significant. Also, on ARM64, the Raytrace (RAY) and Gaussian-Blur (BLUR) seem to be slower: this apparent slowdown is caused by the large number of leftover checks necessary to preserve the execution semantics.

B. Measuring the Impact of Branches in Checks

In the previous section, we established how removing checks results in a significant speedup. However, we do not know whether the speedup comes from the computation of the Boolean check condition or the additional control-flow complexity introduced by these branches also plays a role. We already know that the check branches are rarely taken, as we observed earlier that deoptimizations are very infrequent (Figure 6). The question is how expensive the non-taken branches are, compared to the cost of computing the conditions.

To better characterize the impact of such branches, we modify the code generator in TurboFan so that conditional branches for the checks are not emitted, but without altering the computation of Boolean conditions themselves. Figure 10 presents the relative change of different low-level hardware metrics (collected with `perf`) of the JetStream2 benchmark suite after removing check branches, aggregated by category.

We can make the following observations from these measurements. The first observation is that the number of retired instructions is reduced by 5% and the branch count by 20%. However, this reduction achieves only a modest 1–2% speedup. Despite the 20% reduction of branches, the mispredictions are only reduced by 2–5%: check branches are almost always predicted correctly, and the cost of a correctly predicted branch is mostly negligible. In Figure 10, the reduction of mispredictions is larger on the ARM processor than on X64. This indicates that lowering the total branch count helps the branch predictor improve the accuracy of the remaining branches.

Although removing check branches lowers the instruction count by 5% when, the total number of clock cycles only by 1–2%. On X64, we note an increase in stalled frontend cycles, up to 5% of cycles stall in the frontend. Further analysis with `perf` indicates that benchmarks with increased frontend stalls suffer from increased pressure from stalls in the backend. Although check branches are almost always predicted correctly, their presence is still limiting the CPU backend from filling all instruction slots. Without branches, however, the bottleneck moves closer to the CPU backend.

Overall, removing check branches alone has a minor impact on performance. This result is relevant when designing ISA extensions for JavaScript. Simply hiding these branches (for example, by merging them with other instructions or handling deoptimizations using exceptions) will not provide significant gains. Instead, a more promising direction is the optimization in the computation of check conditions, even more so, if other downstream computations can benefit from such an optimization. We provide an example of such an ISA extension that improves the performance of SMI arithmetic in the next section.

V. EXTENSION OF ARM64 ISA IN V8

SMI arithmetic is a key performance optimization in V8, as it avoids heap accesses for integer values that can be stored in 31 bits. Unlike arithmetic on machine numbers which can be performed within a single instruction, operations on SMIs typically require a sequence of CPU instructions. If SMIs are loaded from memory into a register and the compiler is forced to speculate, a *Not-a-SMI* deoptimization check must be performed (through a `tst` followed by `b.ne`, Figure 3) to check whether the value is really an SMI. The value is then converted into a machine integer by right-shifting it by one bit before the arithmetic operation can be performed. This pattern is also prevalent in array accesses and loops with SMI loop variables. Therefore, SMI arithmetic is a good candidate for HW/SW optimizations. To this end, we propose a set of new ARM64 instructions that optimize loading of SMI values, we call `jsldrsmi`

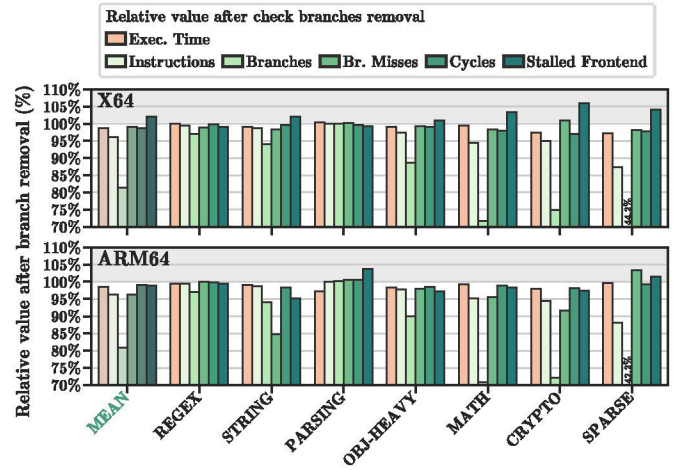


Fig. 10: Removing only the branch instruction from checks does not result in a significant speedup. However, we still see a 5–10% reduction in retired instructions and a 20% reduction in branches. On X64, stalled frontend cycles increase by 3%.

and `jsldrsmi`, as specializations of the load instructions `ldr` and `ldur`. Instead of performing the *Not-a-SMI* check and the *untagging shift* through additional instructions, `jsldr(u)smi` performs all these operations with a single hardware component.

A. Optimizing SMI Arithmetic with Custom Load Instructions

The idea behind our ISA SMI load extension is that integers should only be represented as SMI values in memory, given that SMIs are primarily a memory optimization. When an SMI is loaded into a register, it should be transparently converted into a regular machine integer, allowing the subsequent instructions to directly operate on it. We add six new SMI load instructions, all belonging to the `ld(u)r` family. These instructions support loads with immediate, register, scaled, and unscaled offsets, covering the full set of load instructions that operate on SMIs in V8. Figure 11 shows how the code in Figure 3 is modified with our new instructions. Note the reduction of the code footprint and the lack of explicit deoptimization checks. The additional instructions at the beginning and the end of the code allow us to handle deoptimization without any explicit branch instructions. Instead, we use three new registers: `REG_BA` stores the address of the bailout handler. `REG_PC` and `REG_RE` are written by our SMI load instructions when the deoptimization check fails. `REG_PC` stores the program counter of the failed load instruction. `REG_RE` contains the code of the deoptimization reason, i.e., an 8-bit code that identifies the deoptimization type. The shown ISA extension only supports one deoptimization type, thus, the `REG_RE` is not strictly necessary. We introduce this register to distinguish the deoptimization reason, allowing us to support future check types that have more than one failure reason. If a check fails, execution moves to the bailout handler through an exception triggered during the instruction commit phase, if `REG_RE` is not 0. In turn, the bailout handler uses `REG_PC` and `REG_RE` to identify which check failed and initiate the bailout process through the same sequence of instructions as the unmodified V8. Our extension does not alter the cost of the bailout handler.

Figure 12 illustrates the simple data path extension in the CPU core that implements our SMI load instructions. The *untagging shift* is performed in parallel to the *Not-a-SMI* check, and the target register (e.g., `X0`) is updated concurrently with the special registers `REG_PC` and `REG_RE`. Note that even when loading values into a 64-bit


```

// Global (common) header:
// write bailout_handler into REG_BA
adrp x0, bailout_handler
add x0, x0, :lo12:bailout_handler
msr REG_BA, x0

(start of other code...)

jsldrsmi x5, [sp, #88]    load array index
jsldursmi w4, [x2, #11]   load array size
cmp x5, x4               out-of-bounds check
b.hs #+0x224             deopt if out-of-bounds
lsl x6, x5, #2           prepare for array access
add x4, x6, #0x7
ldr w3, [x3, x4]         perform array access
jsldrsmi x4, [sp, #80]    load operand from stack

(at the end of the function...)

bailout_handler:
mrs x0, REG_PC           x0 ← PC of failed load
mrs x1, REG_RE           x1 ← bailout reason
(other instruction to parse the bailout reason)

```

Fig. 11: Code snippet from Figure 3 updated with our optimized SMI load instructions. They benefit *Not-a-SMI* checks, but also *Out-of-bounds* checks and subsequent arithmetic instructions on SMIs, thanks to our implicit *untagging shift*.

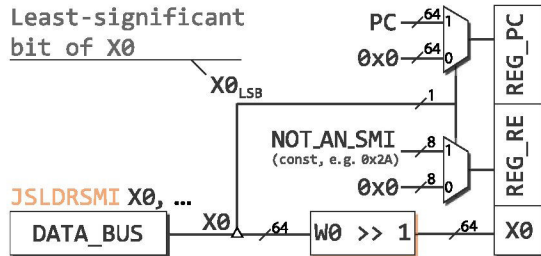


Fig. 12: Block diagram of our SMI load instruction. Even when loading 64-bit values on x registers, we have to perform the arithmetic shift on the corresponding 32-bit w register.

register, such as X0, the *untagging shift* operates on the 32-bit view of the register, as SMIs only contain relevant data in the lower 32 bits. The result of the *Not-a-SMI* check activates two multiplexers, used to update values in REG_PC and REG_RE.

Adding our new instructions to V8 requires several changes to the TurboFan compiler. We modified the ARM64 backend to generate new instructions and introduced new nodes to the SoN graph representation. We made multiple optimization passes aware of the new instructions, e.g., by suppressing the emission of shift instructions and deoptimizations that are no longer necessary. We prototype the SMI load instructions through the cycle-accurate gem5 simulator [19].

B. Performance Analysis of our Extended ISA

We validate our ISA extension on small-scale benchmarks that make use of SMI arithmetic and run with a reasonable time budget in gem5. Each selected benchmark takes around 5 milliseconds on real hardware (using the unmodified ISA), which still translates to one hour or more of simulation time for each benchmark run. Each benchmark runs 10 times, with and without custom SMI instructions. Inputs are fixed and the correctness of the results validated at the end of execution. The main benchmark function of each benchmark (e.g., mmul) is executed multiple times (e.g., 500, for each of the 10 benchmark runs). With the exception of AES2, all benchmarks reach a stable compilation state before we start recording statistics through gem5. Our benchmark suite is composed of computations mainly operating on SMIs. SPMV, SPMM, HASH and BLUR are

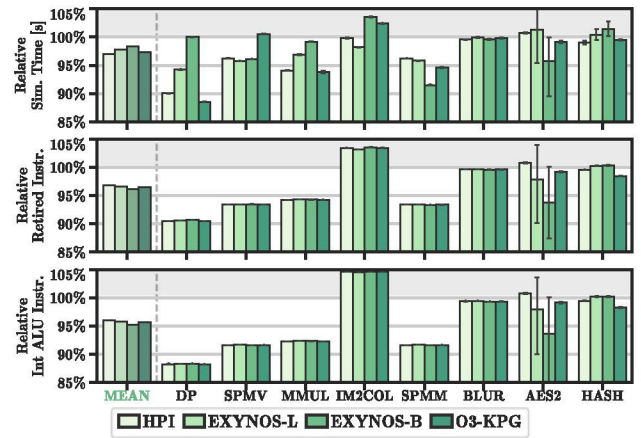


Fig. 13: Our custom SMI load instructions provide a 3 % speedup on average, up to 10 % on simpler CPUs, and a 4 % average reduction of integer ALU instructions.

the same as Section IV, just operating on smaller inputs. AES2 is the encryption step from the AES2 benchmark in JetStream2. DP and MMUL are the dot-product and matrix-multiplication of SMI vectors and matrices, while IM2COL is a preprocessing algorithm used to perform convolutions through matrix multiplications, commonly found in Deep Learning.

We validate our extended ISA on four ARM64 CPUs. Since V8 uses multiple threads to run the compiler and Garbage Collection, we chose CPUs with multiple cores.

- High-Performance In-order (HPI): it represents a modern in-order ARMv8-A cores, with a 4-stage pipeline, L2 caches, 2 cores with 4 GHz, and 2 GB of DRAM [20].
- Exynos-LITTLE: we use an octa-core Exynos CPU (4 big cores at 2.2 GHz, 4 LITTLE cores at 1.8 GHz) derived from [21], with L3 cache and 2 GB of LPDDR3. This configuration pins V8 to the in-order LITTLE cores.
- Exynos-big: identical to Exynos-LITTLE but V8 pinned to the O3 big cores.
- O3-KPG: an O3 server-grade CPU modeled after the ARM CPU in Section IV. It has 4 cores at 2.66 GHz, and 8 GB of DDR4.

From results in Figure 13, we observe an average execution time reduction of 3 % thanks to our extended ISA, up to 10 % for SMI-heavy computations such as DP and SPMM. Although the simpler in-order CPUs show a slightly better average speedup, the complex O3 CPUs can still take advantage of our custom instructions. In some cases (e.g., SPMM, AES2), a better speedup is achievable than on the in-order CPUs. We also observe a 4 % reduction of retired instructions, mostly attributed to the lower number of ALU instructions that are performed with our extended ISA. Through the memory load unit extension in Figure 12, fewer explicit test and shift instructions are necessary. Some speedups, such as SPMV and SPMM on Exynos-big, closely resemble the *SMI* and *Boundary checks* overheads in Figure 4. The AES2 benchmark has higher variance on the Exynos CPU due to TurboFan compilation events happening while we are recording statistics. Additional recompilations also explain the higher run time of the IM2COL benchmarks when using our extended ISA.

By comparing the execution time distribution of the standard and extended ARM64 ISA, we can obtain additional insight into the performance profile of different benchmarks and CPUs (Figure 14). While the performance gap is evident in some cases (e.g., Dot-

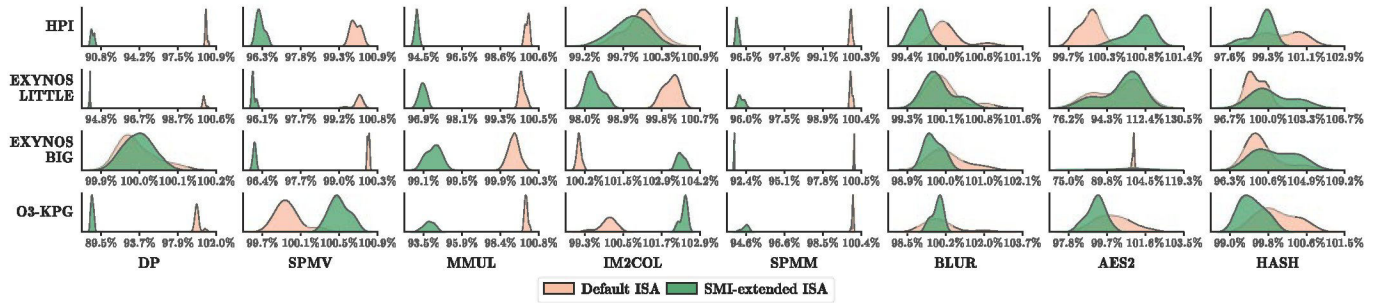


Fig. 14: We can better characterize the performance of our extended ARM64 ISA by looking at execution time distributions for different benchmarks and CPUs. In multiple cases, variance is reduced (e.g., BLUR on all CPUs, or AES2 on O3-KPG). In some cases (e.g., BLUR on the Exynos-big core), this phenomenon results in lower median execution time.

Product (DP) on HPI), in other cases, the difference is less noticeable. While the mean execution time might look unchanged, our extended ISA often provides lower variance, and a better median execution time, such as for BLUR on Exynos-big and AES2 on O3-KPG.

VI. RELATED WORK

Previous analyses of workloads and overheads of JavaScript engines, and also previous proposals of HW/SW codesign techniques, have prevalently focused on X64 [3], [6], [4]. While we also characterize check overheads on X64, we focus our work on ARM64. To the best of our knowledge, we are the first to do so. This result is highly relevant for two reasons. First, ARM is not only the dominant architecture in the mobile space—where JavaScript is extremely common for client-side applications and web browsers, but it is also getting traction in the desktop and server—where platforms such as Electron and Node.js are increasingly popular—with processors such as the Apple M1 and the Ampere Altra. Second, the fixed-length instruction word in a RISC ISA limits the number and size of instruction operands that can be encoded while X64, which supports variable-length instructions, offers more flexibility.

Southern et al. [3] analyzed the check overheads in V8, focusing on X64 processors and estimated that an overhead of 2.2 % and 4.6 % on high and low-performance Intel processors, respectively. These estimates were obtained in 2016 using the old V8 compiler (Crankshaft), and cannot be considered representative of the current state of V8. Indeed, we measured that checks have an average overhead of 8 %, even on server-grade CPUs (Figure 7), significantly more than what was assumed before. We also use a broader benchmark suite that aims to be more representative of real JavaScript workloads and include ARM64 into our analysis. Most importantly, we combine multiple new methodologies to estimate check overheads and analyze results with a thorough statistical analysis, something that was missing from previous research on V8.

HW/SW codesign optimizations for JavaScript also attracted interest in the past. Both Dot et al. [6] and Anderson et al. [22] propose an ISA extension for X64 that targets the optimization of checks in JavaScript. Our ISA extension, while indirectly optimizing *Not-a-SMI* and *Boundary Checks*, addresses a more general problem of dynamic languages: how to support efficient data representations (in our case, SMIs) for types only available at run time, without introducing overheads during memory operations. Other techniques use dynamic information tracing to remove, for example, the generation of checks that are known to never fail at run time [23], [4].

The ARMv8.3-A ISA already contains a specific instruction for JavaScript, `FJCVTZS`, which converts double-precision floating-point

numbers to 32-bit integers. It can be configured to trigger a floating-point exception if the conversion fails. V8 already makes use of this instruction; however, it does not handle conversion errors. This instruction is orthogonal to our SMI optimization, as our goal is to improve the handling of SMIs given that V8 avoid using floating point numbers whenever possible.

Research interest in dynamic languages and HW/SW codesign goes beyond JavaScript, V8, and ARM. For example, similar analyses on Python have determined that function calls and the cache hierarchy are a major source of run-time overhead [24], [25], and that the PyPy JIT compiler, just like V8, shows a significant performance difference between the start-up and steady-state performance [26]. Other researchers have shown how custom hardware can improve Garbage Collection performance by up to 60 % in Python and V8 [5], and by 3× in Java when running on an in-order CPU [27].

An instruction set extension that adds hardware support for JITs to RISC-V, the J Extension, is currently being drafted. Features that are being discussed are hardware support for pointer tagging.

VII. CONCLUSION

We characterized how JavaScript code running in V8, one of the widely used JavaScript engine, spends about 8 % of its time in deoptimization checks. This value is 2–4× higher than what earlier studies reported on an earlier version of V8. This difference motivated us to perform an up-to-date analysis of check overheads in V8. Most importantly, we find that this overhead justifies novel research on HW/SW codesign optimizations. Through our ISA extension, we show how to improve the performance of a modern JavaScript engine by up to 10 %, on SMI-intensive computations.

While we focused on the commonly-occurring SMI arithmetic, we believe that similar optimizations are possible for other checks, e.g., map and boundary checks. These techniques are not limited to V8. Other JavaScript engines, such as WebKit’s JavaScriptCore and Mozilla’s SpiderMonkey, also have multi-tier JIT compilers that perform similar type speculations. Although the implementation details are different, our methodology to estimate check overheads and our ISA extension could be applied to these engines.

Furthermore, opportunities in the context of HW/SW codesign are not limited to JavaScript but exist for dynamic languages in general. One area besides optimizing checks is hardware-assisted and cache-based type-tracing and accelerating the collection of type feedback. Also possible is the hardware-acceleration of specific and frequently used *builtin functions* (e.g., *string equality*), which we have measured to take up to 8 % of the execution time in string-intensive benchmarks.

REFERENCES

- [1] W³Techs Web Technology Surveys, “Usage statistics of javascript as client-side programming language on websites,” w3techs.com/technologies/details/cp-javascript, retrieved on 2021-06-23.
- [2] Internet Live Stats, “Total number of websites,” internetlivestats.com/total-number-of-websites, retrieved on 2021-06-23.
- [3] G. Southern and J. Renau, “Overhead of deoptimization checks in the v8 javascript engine,” in *2016 IEEE International Symposium on Workload Characterization (IISWC)*. IEEE, 2016, pp. 1–10.
- [4] G. Dot, A. Martínez, and A. González, “Removing checks in dynamically typed languages through efficient profiling,” in *2017 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. IEEE, 2017, pp. 257–268.
- [5] M. Ismail and G. E. Suh, “Hardware-software co-optimization of memory management in dynamic languages,” *ACM SIGPLAN Notices*, vol. 53, no. 5, pp. 45–58, 2018.
- [6] G. Dot, A. Martínez, and A. González, “Analysis and optimization of engines for dynamically typed languages,” in *2015 27th International Symposium on Computer Architecture and High Performance Computing (SBAC-PAD)*. IEEE, 2015, pp. 41–48.
- [7] L. Swirski, “Sparkplug — a non-optimizing javascript compiler,” v8.d ev/blog/sparkplug, 2021-05-27, retrieved on 2021-06-22.
- [8] C. Click and M. Paleczny, “A simple graph-based intermediate representation,” *ACM Sigplan Notices*, vol. 30, no. 3, pp. 35–49, 1995.
- [9] “V8 torque user manual,” v8.dev/docs/torque, retrieved on 2021-06-22.
- [10] J. Franco, “An internship on laziness: lazy unlinking of deoptimized functions,” v8.dev/blog/lazy-unlinking, 2017-10-04, retrieved on 2021-03-31.
- [11] J. Sevcik, “Deoptimization in v8,” docs.google.com/presentation/d/1Z6oCocRASCFtqGq1GC01jbULDGS-w-nzxkbVF7Up0u0/htmlpresent, 2016-12-09, retrieved on 2021-06-23.
- [12] S. Barati and M. Saboff, “Introducing the jetstream 2 benchmark suite,” webkit.org/blog/8685/introducing-the-jetstream-2-benchmark-suite, 2019-03-27, retrieved on 2021-03-31.
- [13] C. Developers, “Octane javascript benchmark suite,” github.com/chromium/octane, retrieved on 2021-03-31.
- [14] Mozilla, “Kraken javascript benchmark,” wiki.mozilla.org/Kraken, 2021-02-22, retrieved on 2021-03-31.
- [15] WebKit, “Sunspider 1.0.2 javascript benchmark,” webkit.org/perf/sunspider/sunspider.html, retrieved on 2021-03-31.
- [16] E. W. Weisstein, “Bonferroni correction,” <https://mathworld.wolfram.com/BonferroniCorrection.html>, 2004.
- [17] E. S. Pearson, R. B. D’Agostino, and K. O. Bowman, “Tests for departure from normality: Comparison of powers,” *Biometrika*, vol. 64, no. 2, pp. 231–246, 1977.
- [18] F. Wilcoxon, “Individual comparisons by ranking methods,” in *Breakthroughs in statistics*. Springer, 1992, pp. 196–202.
- [19] N. Binkert, B. Beckmann, G. Black, S. K. Reinhardt, A. Saidi, A. Basu, J. Hestness, D. R. Hower, T. Krishna, S. Sardashti *et al.*, “The gem5 simulator,” *ACM SIGARCH computer architecture news*, vol. 39, no. 2, pp. 1–7, 2011.
- [20] A. Tousi and C. Zhu, “Arm research starter kit: System modeling using gem5,” 2017.
- [21] A. Butko, F. Bruguier, A. Gamatié, G. Sassatelli, D. Novo, L. Torres, and M. Robert, “Full-system simulation of big, little multicore architecture for performance and energy exploration,” in *2016 IEEE 10th international symposium on embedded multicore/many-core systems-on-chip (mcso)*. IEEE, 2016, pp. 201–208.
- [22] O. Anderson, E. Fortuna, L. Ceze, and S. Eggers, “Checked load: Architectural support for javascript type-checking on mobile processors,” in *2011 IEEE 17th International Symposium on High Performance Computer Architecture*. IEEE, 2011, pp. 419–430.
- [23] A. Gal, B. Eich, M. Shaver, D. Anderson, D. Mandelin, M. R. Haghighat, B. Kaplan, G. Hoare, B. Zbarsky, J. Orendorff *et al.*, “Trace-based just-in-time type specialization for dynamic languages,” *ACM Sigplan Notices*, vol. 44, no. 6, pp. 465–478, 2009.
- [24] B. Ilbeyi, C. F. Bolz-Tereick, and C. Batten, “Cross-layer workload characterization of meta-tracing jit vms,” in *2017 IEEE International Symposium on Workload Characterization (IISWC)*. IEEE, 2017, pp. 97–107.
- [25] M. Ismail and G. E. Suh, “Quantitative overhead analysis for python,” in *2018 IEEE International Symposium on Workload Characterization (IISWC)*. IEEE, 2018, pp. 36–47.
- [26] A. Crapé and L. Eeckhout, “A rigorous benchmarking and performance analysis methodology for python workloads,” in *2020 IEEE International Symposium on Workload Characterization (IISWC)*. IEEE, 2020, pp. 83–93.
- [27] M. Maas, K. Asanović, and J. Kubiawicz, “A hardware accelerator for tracing garbage collection,” in *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2018, pp. 138–151.